SolariNet migration runbook

Candidate procedure · refreshed 21 October 2026 · monitoring DB and SoR

> **Production gate:** Current live databases have no `solari_migration_ledger`. The runner intentionally refuses them. Do not stamp their history from table presence alone; audit a backup and compare the actual schema against every historical migration first.

1. What the runner does

`tools/migrate.py` applies full file IDs in order, including both distinct `016_*` monitoring migrations. It stores each SHA-256 checksum and an `applying` / `applied` state in the target database. Before writing, it inventories tables, views, routines, and scheduled events; it refuses edited or missing historical files, gaps, interrupted runs, and schemas with objects but no ledger. MariaDB DDL is not transactional; an interrupted file requires inspection rather than automatic replay.

Stream	Initial input	Files proven on scratch DB
Monitoring	`db/migrations/001_baseline.sql` through `033_redact_queued_decommission_tokens.sql`	34 applied, including both 016 files and migration 033
SoR	`netdb/sor/schema.sql`, then its numbered migrations	8 applied through 023

2. Clean installation

Create each new empty database with the canonical character set and collation using an account permitted to create databases. The runner checks this default before inventory or writes, because tables that inherit database defaults must be reproducible across MariaDB hosts. It refuses another default; it does not alter an existing database. Supply migration credentials through a protected MariaDB option file, a local Unix socket account, or a process environment managed outside git. Run from any directory; the script resolves SQL paths from its own location.

```
mariadb -e 'CREATE DATABASE NEW_MONITORING_DB CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci; CREATE DATABASE NEW_SOR_DB CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;'
python3 tools/migrate.py --database monitoring --db-name NEW_MONITORING_DB --check
python3 tools/migrate.py --database monitoring --db-name NEW_MONITORING_DB
python3 tools/migrate.py --database monitoring --db-name NEW_MONITORING_DB --check

python3 tools/migrate.py --database sor --db-name NEW_SOR_DB --check
python3 tools/migrate.py --database sor --db-name NEW_SOR_DB --defaults-file /path/
to/protected.cnf
```

The final `--check` should show no `PENDING` files.

Regenerate reference snapshots

Each command creates a temporary database, applies the full ledgered stream, and writes a deterministic schema-only artifact.

```
python3 tools/export_schema.py --database monitoring
python3 tools/export_schema.py --database sor
```

Snapshot export requires permission to create and drop a temporary database. CI compares both snapshots with clean migration runs. Snapshots are reference artifacts only; install through the ledgered migrator. `solariServer --init-db` remains retired.

3. Existing databases and failure recovery

Live SoR connection recheck (2026-10-21): a read-only connection from Xenon through the deployed SoR sync service configuration reports Cesium MariaDB `12.3.2`, 41 SoR base tables, 24 triggers, and no `solari_migration_ledger`. The query emitted schema metadata only; it did not read application rows or change the database. This confirms the existing adoption gate remains current.

Before preparing an adoption record, compare the current schema to the candidate snapshot with the read-only auditor. It makes no schema changes and excludes the runner's own ledger from the comparison:

```
python3 tools/audit_schema.py --database monitoring --db-name solarinet \
  --defaults-file /path/to/protected.cnf
python3 tools/audit_schema.py --database sor --db-name sor \
  --defaults-file /path/to/protected.cnf
```

An exact match compares tables, views, triggers, routines, and events, while normalizing account-specific definers and live next-ID counters. It verifies schema shape only. A unified

diff identifies object-definition drift for review; it does not prove historical migration execution, validate data or grants, or approve stamping a ledger. This conservative comparison is expected to report differences for the currently documented production schemas.

1. Back up the database and record the table, column, index, trigger, and view definitions. Confirm which migrations ran, including the two monitoring files numbered 016.
2. Compare the actual schema with the full ordered file set. Resolve any drift through a reviewed forward migration. Only then prepare an explicit adoption record with matching immutable file checksums.
3. If the runner reports `applying`, inspect which statements in that file committed. Repair or complete the partial DDL, then record the verified result. Do not delete a ledger row merely to retry the file.
4. Re-run `--check` before applying pending files. Stop if the ledger checksum differs from the checked-out file.

4. Evidence and limits

Migration-runner clean-install verification (2026-10-21): using temporary, uniquely named schemas on Xenon's MariaDB 11.8.6, the actual `tools/migrate.py` runner applied the complete monitoring stream and SoR stream, reapplied both without pending work, and passed `tools/audit_schema.py` for each. The schemas used the required `utf8mb4/utf8mb4_unicode_ci` defaults and were dropped after verification; a follow-up metadata query confirmed neither remains. The tool regression suite passed 21/21. This verifies fresh-install and repeat-apply behavior on the local MariaDB version; it does not authorize or validate adoption of the existing Xenon/Cesium schemas.

Observed in scratch databases: A fresh MariaDB 11.8.6 validation on 2026-10-21 applied all 34 monitoring and 8 SoR migration files; clean rechecks produced no pending entries. The generated monitoring and SoR schema snapshots matched byte-for-byte, then restored into separate clean databases with identical table/view sets and SoR trigger set. The monitoring snapshot includes migrations through 033 queued-decommission-token redaction; the SoR snapshot has 45 tables and 24 triggers, including the CDC contract from migrations 022–023. The runner refused an untracked populated database and a simulated interrupted ledger row in earlier tests. A focused regression verifies an out-of-order ledger is rejected before any schema-mutating command. The migrator also refuses a database whose default character set or collation is not canonical utf8mb4/utf8mb4_unicode_ci before inventory or writes. Separate disposable MariaDB 11.8.6 runs matched both read-only audits, detected injected drift, and rejected wrong-collation databases before creating objects. Run local tool tests with `python3 -m unittest discover -s tools -p 'test_*.py'`. The fresh 2026-10-21 decommission test database is disposable scratch state.

Read-only production schema comparison (initial, 2026-10-20): Xenon runs MariaDB 11.8.6; `solarinet` has 29 base tables and no migration ledger. The then-current candidate had 31 tables. An initial comparison identified missing `ctlRequest`/`reportReceipt` tables and seven fields added by migrations 022–024: `enrollment.nodeId`, `enrollment.issuedCertPem`, `enrollment.enrollTokenHash`, `enrollment.tokenExpiresAt`, `enrollment.certAckAt`, `node.leafCertFp`, and `node.leafCertSerial`. It also lacks `uq_enrollment_token_hash` and the `panelStateSingleton` check; it has an extra `maintenanceWindow.idx_window_ip` index. A read-only row check found one legacy approved enrollment with no CSR, certificate fingerprint, or matching node by exact host and role. Do not auto-backfill that record.

Cesium runs MariaDB 12.3.2; `sor` has 41 base tables and four views, with no migration ledger. Compared with the 45-table candidate, it lacks the four tables from migration 021 (`functions`, `entity_functions`, `function_incompatibility`, `inventory_snapshots`; 51 columns). The live `sor_outbox`/`sor_sync_state` tables and 21 CDC triggers now have canonical migration sources in 022–023; the three inventory-allocation triggers remain in 014. Production still has no migration ledger. The existing `ip_addresses` table lacks two migration-021 checks, and `fk_ip_entity` still uses `ON DELETE SET NULL` where the candidate requires `RESTRICT`. Read-only data checks found no invalid primary flags, active primary address without an entity, or orphan entity reference. Shared table/column/index metadata otherwise matched; view definitions match after normalizing the database qualifier.

2026-10-21 read-only recheck: full schema-only dumps from Xenon and Cesium were compared with the current generated snapshots after normalizing definers, live next-ID counters, and dump delimiter formatting. Xenon still has 29 base tables versus 33 in the candidate. It lacks `ctlRequest`, `decommissionChallenge`, `discoveryScanJob`, and `reportReceipt`; this refresh supersedes earlier table counts. In addition to the previously listed enrollment/node gaps, it lacks `uq_enrollment_token_hash` and the `panelStateSingleton` check, and has the extra `maintenanceWindow.idx_window_ip` index. Eight object blocks differ overall. Cesium still has 41 base tables versus 45 in the candidate and lacks `entity_functions`, `function_incompatibility`, `functions`, and `inventory_snapshots`. Its `ip_addresses` table lacks generated columns `live_flag`/`primary_entity_flag`, unique indexes `uq_ip_addr_live`/`uq_ip_addr_primary_entity`, and checks `ck_ip_addr_is_primary`/`ck_ip_addr_primary_has_entity`; `fk_ip_entity` still uses `ON DELETE SET NULL` instead of candidate `RESTRICT`. Eleven object blocks differ overall. The four live SoR view definitions were queried directly from `information_schema.VIEWS` and match candidate logic after removing database qualifiers. The leading `1 AS column` sections in dump output are view-column placeholders, not the definitions. No production SQL was applied. Neither database has a migration ledger.

Read-only collation check (2026-10-21): Xenon `solarinet` defaults to `utf8mb4_unicode_ci` and its 29 base tables use that collation. Cesium `sor` has the same database default; 39 base tables use `utf8mb4_unicode_ci` and two use `utf8mb4_uca1400_ai_ci`. These are metadata checks only; no production schema was changed.

The comparison covered table options, column definitions, indexes, constraints, foreign-key rules, check clauses, triggers, and views. It did not establish historical migration execution, compare grants or seed/application data, or test backups/restores. Preserve the extra Xenon index pending query-plan review. Do not adopt or stamp either database from this comparison. The production enrollment API also cannot support the candidate token flow or persist issued-leaf identity until the missing columns and index are applied.

Backup/restore evidence · 2026-10-21: Cesium's `solari-backup@cesium.timer` is enabled and active. The latest `sor-2026-10-21.sql.gz` archive passed `gzip -t` and was streamed into an isolated local MariaDB database; restore succeeded, all 41 base tables passed `mariadb-check`, and four views plus 24 triggers were present. Xenon's `solari-backup@xenon.timer`, its documented config/script paths, and a matching dump in the checked backup roots were absent. No production data was restored or changed.

Observed production guard: On 2026-10-21, `sudo -n python3 tools/migrate.py --database monitoring --db-name solarinet --check` on Xenon returned `untracked nonempty database` and stopped before creating a ledger or applying migration SQL. This was read-only; no production schema was changed.

Unverified: Historical migration attribution, a safe resolution for the orphaned approved enrollment, a verified Xenon monitoring backup/restore, staged schema upgrades, live application compatibility, grants, and remote CI execution remain release gates. The candidate runner does not automatically adopt legacy schemas.

Standalone HTML manual with inline CSS and SVG. The matching PDF is generated from this file. No external assets or scripts are required.